

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4

Total Marks 20

Duration :60 Minutes

Annexure A Mock

Interview

Code of Conduct:

I P.Jyothika(192311156) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Priyanga.B

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	

Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	
-----------------------	----	--	---	------------------------------------	---------------------------------	--

Image & Video Processing

1.A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.

Problem Analysis

Frame drops almost always mean the per-frame processing time is longer than the interval at which new frames arrive. In a naive design, capture and processing run on the same thread in one loop — while a frame is being processed, the capture buffer keeps filling up, and once it overflows, the OS/driver silently discards frames. So the root cause is a blocking, single-threaded pipeline that cannot keep pace with the incoming frame rate.

Redesign

- Decouple capture from processing using a producer–consumer model: a dedicated capture thread continuously reads frames into a bounded queue, while a separate processing thread consumes frames independently — the capture loop is never blocked waiting for processing.
- Use a bounded queue with a 'keep-latest' drop policy: if the queue is full, discard the oldest unprocessed frame rather than blocking the producer, so the system stays close to real time instead of building up latency.
- Reduce per-frame cost: downscale frame resolution, use a lighter model (e.g. MobileNetSSD instead of a heavy CNN), or only run the expensive model every Nth frame.
- Use hardware acceleration where available (cv2.dnn with a CUDA/OpenVINO backend) to cut inference time directly.
- For playback smoothness specifically, decouple the display/output rate from the processing rate — if a new processed frame isn't ready in time, briefly re-display the last processed frame rather than stalling, so playback appears continuous even if analysis lags slightly.

Trade-offs

- Multithreading improves throughput but adds complexity and some memory overhead for the queue.
- Dropping frames keeps the system real-time but sacrifices completeness — some events may be missed entirely.

- Downscaling/lighter models trade a measure of accuracy for speed.
- Hardware acceleration gives large speed gains but adds a hardware/software dependency.

Conclusion: A producer–consumer architecture with a bounded, drop-oldest queue, combined with lighter/accelerated processing, is the standard fix — it keeps the capture side unblocked and trades a small, controlled amount of completeness for a smooth, real-time playback experience.

2. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.

Problem Analysis

A naive approach reads and fully decodes every frame with `cap.read()`, then simply ignores four out of five of them. That still pays the full decoding cost for every frame, wasting most of the saved time. The efficient version should avoid the expensive decode step for frames that will be discarded anyway.

Design

```
frameIndex = 0
WHILE cap.IS_OPENED() DO
    grabbed = cap.GRAB()      // advances pointer, skips full decode
    IF NOT grabbed THEN
        BREAK
    END IF
    IF frameIndex MOD 5 == 0 THEN
        ret, frame = cap.RETRIEVE() // full decode ONLY for needed frame
        ProcessFrame(frame)    END IF
        frameIndex = frameIndex + 1
    END WHILE
```

Justification

`cap.grab()` advances the decoder to the next frame without performing the full colour-space conversion/decode that `cap.retrieve()` (or the combined `cap.read()`) does — it is significantly cheaper. By calling `grab()` on every frame but `retrieve()` (full decode) only when `frameIndex % 5 == 0`, roughly 80% of the decoding cost is avoided, giving a large performance gain over decoding every frame and discarding most of them. The accuracy trade-off is that events occurring between the sampled frames are missed — acceptable for applications like periodic monitoring or thumbnail generation, but risky if fast-moving objects could appear and disappear within a 5-frame window, in which case a smaller skip interval or motion-triggered sampling would be safer.

Conclusion: Using grab()-then-conditional-retrieve() to skip frames is far more efficient than decoding every frame and discarding most of them, and is justified whenever the application does not need frame-level temporal precision.

3. A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.

Problem Analysis

An image is essentially a single frame, and a video is a sequence of frames over time — so at the core-processing level (detection, filtering, analysis) there is no real difference between the two. The only real difference is how the input is obtained and how the output is delivered. So the right design is to isolate that difference behind a common interface, and write the actual processing logic exactly once.

Generalized Design

```
FUNCTION GetFrameSource(inputPath)
  IF FileType(inputPath) == IMAGE THEN
    RETURN SingleFrameIterator(LOAD_IMAGE(inputPath))
  ELSE IF FileType(inputPath) == VIDEO THEN
    RETURN VideoCapture(inputPath) // multi-frame iterator
  END IF
END FUNCTION

MAIN:
  frameSource = GetFrameSource(inputPath)
  isVideo     = (FileType(inputPath) == VIDEO)
  WHILE frameSource.HAS_NEXT() DO
    frame = frameSource.NEXT()
    result = ProcessFrame(frame) // SAME function for image & video
    IF isVideo THEN
      WRITE_TO_VIDEO(result, videoWriter)
    ELSE
      SAVE_IMAGE(result, outputPath)
    END IF
  END WHILE
```

How It Handles Both Cases

A thin adapter layer (GetFrameSource) normalizes any input — single image or video file — into a uniform frame-iterator interface: a static image simply becomes an iterator that yields one frame and stops, while a video is the natural multi-frame iterator OpenCV already provides via VideoCapture. The ProcessFrame() function, which contains all the actual computer-vision logic, has no idea whether its input came from an image or a video frame — it just receives a frame and

returns a result. Only the outermost layer (output handling) branches on input type, to decide between writing a single image file or streaming frames into a video writer. This is a clean separation of concerns: input handling, core processing, and output handling are three independent, swappable modules, so the same tested pipeline logic is reused for both cases instead of duplicated.

Conclusion: Wrapping both input types behind a common frame-iterator interface and keeping the processing function source-agnostic lets one pipeline correctly and efficiently handle images and videos without code duplication.

4. You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.

Problem Analysis

Saving every frame wastes storage and produces many near-duplicate frames during static scenes. This is essentially a key-frame / scene-change detection problem: we need a numeric measure of how much a frame differs from what has already been captured, and a threshold that decides whether that difference is 'significant' enough to save.

Design

```
lastSavedFrame = NULL
FOR EACH frame IN video DO
    grayFrame = CONVERT_TO_GRAYSCALE(frame) // cheaper, noise-tolerant comparison
    IF lastSavedFrame IS NULL THEN
        SAVE_FRAME(frame)
        lastSavedFrame = grayFrame
        CONTINUE
    END IF
    diff = ABS_DIFF(grayFrame, lastSavedFrame)
    changeScore = COUNT_PIXELS_ABOVE(diff, pixelThreshold) / TOTAL_PIXELS
    IF changeScore > SIGNIFICANCE_THRESHOLD THEN
        SAVE_FRAME(frame)
        lastSavedFrame = grayFrame // reference updates only on save
    END IF
END FOR
```

Decision Criteria and Justification

- Comparison is against the last SAVED frame, not simply the previous frame — this stops slow, gradual drift from being saved one tiny step at a time, since each new candidate is judged against what has actually already been captured.

- Grayscale conversion plus a per-pixel intensity-difference threshold (`cv2.absdiff`) is computationally cheap, making it practical to scan an entire long video.
- The fraction of changed pixels above the threshold — rather than a raw pixel count — normalizes the metric across resolutions and makes the significance threshold easier to tune.
- For applications needing a more perceptually accurate measure (e.g. resistant to minor lighting flicker), the same architecture can swap in SSIM (Structural Similarity Index) or histogram comparison (`cv2.compareHist`) in place of raw pixel differencing, at a higher computational cost.
- A minimum-gap rule (e.g. don't save again within K frames) can optionally be added to avoid saving a burst of frames during one continuous fast-changing event.

Conclusion: Comparing each frame against the last saved reference frame using a lightweight difference metric, gated by a significance threshold, captures meaningful scene changes while discarding redundant near-duplicate frames — achieving the storage and relevance goals of intelligent key-frame extraction.